

Análisis de Lenguajes de Programación

Semántica Operacional

7 de Septiembre de 2026

Lenguaje imperativo simple

Veremos la semántica operacional de un lenguaje imperativo simple, donde modelaremos los efectos laterales de:

- ▶ estado
- ▶ errores
- ▶ E/S

Sintaxis

$ie := nv$

| var

| $- ie$

| $ie + ie$

| $ie -_b ie$

| $ie \times ie$

| $ie \div ie$

$be := true \mid false$

| $ie = ie$

| $ie < ie$

| $ie > ie$

| $be \wedge be$

| $be \vee be$

| $\neg be$

$comm := skip \mid var := ie$

| $comm ; comm$

| $if\ be\ then\ comm\ else\ comm$

| $while\ be\ comm$

- El lenguaje solo contiene variables enteras. La noción de estado se puede modelar con la siguiente definición:

$$\Sigma = Var \rightarrow nv$$

Es decir, un estado es una función total entre identificadores y enteros.

- Para modificar el estado utilizaremos la siguiente notación:

$[\sigma \mid v : n]$ que representa un estado que coincide con σ en todas las variables salvo posiblemente en v , donde tiene asignado n .

Relaciones de evaluación de paso grande

- ▶ Para expresiones enteras la relación tiene este tipo:

$$\Downarrow_i \in (ie \times \Sigma) \times nv$$

- ▶ Regla para variables:

$$\frac{}{(v, \sigma) \Downarrow_i \sigma(v)} \quad (\text{VAR})$$

- ▶ Las demás reglas se extienden con un estado, por ejemplo:

$$\frac{(e_0, \sigma) \Downarrow_i n_0 \quad (e_1, \sigma) \Downarrow_i n_1}{(e_0 + e_1, \sigma) \Downarrow_i n_0 + n_1} \quad (\text{ADD})$$

Evaluación para expresiones booleanas

- La relación de evaluación tiene este tipo:

$$\Downarrow_b \in (be \times \Sigma) \times bv$$

- Las reglas se extienden con un estado, por ejemplo:

$$\frac{(e_0, \sigma) \Downarrow_b b_0 \quad (e_1, \sigma) \Downarrow_b b_1}{(e_0 \wedge e_1, \sigma) \Downarrow_b b_0 \wedge b_1} \quad (\text{AND})$$

$$\frac{(e_0, \sigma) \Downarrow_i n_0 \quad (e_1, \sigma) \Downarrow_i n_1}{(e_0 = e_1, \sigma) \Downarrow_b n_0 = n_1} \quad (\text{EQ})$$

Evaluación para comandos

La relación de evaluación tiene este tipo:

$$\Rightarrow \in (comm \times \Sigma) \times \Sigma$$

$$\frac{(e, \sigma) \Downarrow_i n}{(x := e, \sigma) \Rightarrow [\sigma \mid x : n]} \text{ (ASS)}$$

$$\frac{}{(\text{skip}, \sigma) \Rightarrow \sigma} \text{ (SKIP)}$$

$$\frac{(c_0, \sigma) \Rightarrow \sigma' \quad (c_1, \sigma') \Rightarrow \sigma''}{(c_0; c_1, \sigma) \Rightarrow \sigma''} \text{ (SEQ)}$$

$$\frac{(e, \sigma) \Downarrow_b \text{true} \quad (c_1, \sigma) \Rightarrow \sigma'}{(\text{if } e \text{ then } c_1 \text{ else } c_2, \sigma) \Rightarrow \sigma'} \text{ (IF-T)}$$

$$\frac{(e, \sigma) \Downarrow_b \text{false} \quad (c_2, \sigma) \Rightarrow \sigma'}{(\text{if } e \text{ then } c_1 \text{ else } c_2, \sigma) \Rightarrow \sigma'} \text{ (IF-F)}$$

$$\frac{(e, \sigma) \Downarrow_b \text{true} \quad (c, \sigma) \Rightarrow \sigma' \quad (\text{while } e \text{ } c, \sigma') \Rightarrow \sigma''}{(\text{while } e \text{ } c, \sigma) \Rightarrow \sigma''} \quad (\text{WHILE-T})$$

$$\frac{(e, \sigma) \Downarrow_b \text{false}}{(\text{while } e \text{ } c, \sigma) \Rightarrow \sigma} \quad (\text{WHILE-F})$$

Evaluación de paso chico para comandos

- La relación tiene el siguiente tipo:

$$\rightsquigarrow \in (comm \times \Sigma) \times (comm \times \Sigma)$$

- Si la ejecución termina lo hace en una configuración de la forma $(skip, \sigma)$, para algún σ .
- Las ejecuciones de la asignación y `skip` terminan en un paso, son similares a la de paso grande.
- Secuenciamiento:

$$\frac{(c_0, \sigma) \rightsquigarrow (c'_0, \sigma')}{(c_0; c_1, \sigma) \rightsquigarrow (c'_0; c_1, \sigma')} \quad (SEQ_1)$$

$$\frac{}{(skip; c_1, \sigma) \rightsquigarrow (c_1, \sigma)} \quad (SEQ_2)$$

Evaluación de paso chico para comandos

- Condicionales (mantenemos evaluación de paso grande para expresiones):

$$\frac{e \Downarrow_b \text{true}}{(\text{if } e \text{ then } c_0 \text{ else } c_1, \sigma) \rightsquigarrow (c_0, \sigma)} \quad (\text{IF-T})$$

$$\frac{e \Downarrow_b \text{false}}{(\text{if } e \text{ then } c_0 \text{ else } c_1, \sigma) \rightsquigarrow (c_1, \sigma)} \quad (\text{IF-F})$$

Evaluación de paso chico para comandos

► Bucles:

$$\frac{e \Downarrow_b \text{false}}{(\text{while } e \text{ } c, \sigma) \rightsquigarrow (\text{skip } , \sigma)} \quad (\text{WHILE-F})$$

$$\frac{e \Downarrow_b \text{true}}{(\text{while } e \text{ } c, \sigma) \rightsquigarrow (c; \text{while } e \text{ } c, \sigma)} \quad (\text{WHILE-T})$$

Agregamos errores

- ▶ Agregamos un valor que representa un error: **err_i**
- ▶ Las expresiones enteras que fallen devolverán este valor.
- ▶ La relación de evaluación tendrá este tipo:

$$\Downarrow_i \in (ie \times \Sigma) \times (nv \cup \{ \mathbf{err}_i \})$$

- ▶ Agregamos reglas que generan errores:

$$\frac{(e_0, \sigma) \Downarrow_i n_0 \quad (e_1, \sigma) \Downarrow_i 0}{(e_0 \div e_1, \sigma) \Downarrow_i \mathbf{err}_i} \quad (\text{DIV0})$$

Agregamos errores

- Distinguimos las divisiones que no generan error.

$$\frac{(e_0, \sigma) \Downarrow_i n_0 \quad (e_1, \sigma) \Downarrow_i n_1 \quad n_1 \neq 0}{(e_0 \div e_1, \sigma) \Downarrow_i n_0 \div n_1} \quad (\text{DIV-1})$$

- Agregamos reglas para propagar el error:

$$\frac{(e_0, \sigma) \Downarrow_i \mathbf{err}_i}{(e_0 \div e_1, \sigma) \Downarrow_i \mathbf{err}_i} \quad (\text{DIV-2})$$

$$\frac{(e_0, \sigma) \Downarrow_i n_0 \quad (e_1, \sigma) \Downarrow_i \mathbf{err}_i}{(e_0 \div e_1, \sigma) \Downarrow_i \mathbf{err}_i} \quad (\text{DIV-3})$$

Agregamos errores

- ▶ Agregamos un valor que representa un error booleano: **err_b**
- ▶ La relación de evaluación tendrá este tipo:

$$\Downarrow_b \in (be \times \Sigma) \times (bv \cup \{\mathbf{err}_b\})$$

- ▶ Modificamos las reglas para generar y propagar errores booleanos.

$$\frac{(e_0, \sigma) \Downarrow_i \mathbf{err}_i}{(e_0 = e_1, \sigma) \Downarrow_b \mathbf{err}_b} \quad (\text{EQ-0})$$

Agregamos errores

- Modificamos la relación para comandos:

$$\rightsquigarrow \in (comm \times \Sigma) \times ((comm \cup \{\mathbf{err}_c\}) \times \Sigma)$$

- Ejemplos de algunas reglas:

$$\frac{(e, \sigma) \Downarrow_i \mathbf{err}_i}{(x := e, \sigma) \rightsquigarrow (\mathbf{err}_c, \sigma)} \quad (\text{Ass-0})$$

$$\frac{(c_0, \sigma) \rightsquigarrow (\mathbf{err}_c, \sigma')}{(c_0 ; c_1, \sigma) \rightsquigarrow (\mathbf{err}_c, \sigma')} \quad (\text{Seq-0})$$

- ▶ Agregamos etiquetas a las transiciones entre configuraciones, donde:
 - ▶ $n?$ indicará la entrada de un entero n
 - ▶ $n!$ indicará la salida de un entero n
- ▶ La relación de evaluación tendrá este tipo:

$$\rightsquigarrow \in (comm \times \Sigma) \times I \times ((comm \cup \{\mathbf{err}_c\}) \times \Sigma)$$

donde $I := nv? \mid nv! \mid \tau$

- ▶ La etiqueta τ representa la transición silenciosa, escribimos $x \rightsquigarrow y$ en lugar de $x \xrightarrow{\tau} y$

Agregamos E/S

- ▶ Extendemos la sintaxis de los comandos:

$$comm := \dots \mid \text{input } var \mid \text{print } ie$$

- ▶ Agregamos reglas semánticas para estos comandos:

$$\frac{}{(\text{input } v, \sigma) \xrightarrow{n?} (\text{skip}, [\sigma | v : n])} \quad (\text{INPUT})$$

$$\frac{(e, \sigma) \Downarrow_i n}{(\text{print } e, \sigma) \xrightarrow{n!} (\text{skip}, \sigma)} \quad (\text{PRINT})$$

- ▶ Reescribimos las transiciones de las reglas con una etiqueta arbitraria l .

$$\frac{(c_0, \sigma) \xrightarrow{l} (c'_0, \sigma')}{(c_0; c_1, \sigma) \xrightarrow{l} (c'_0; c_1, \sigma')} \quad (\text{SEQ}_1)$$

- ▶ Theories of Programming Languages. John Reynolds.
(Capítulo 6)